

LAB SHEET

06026212 DATA WAREHOUSING

สัปดาห์ที่ 8: Granularity, Aggregation & Summary Table

OLAP: Roll-up, Drill-down และ CUBE ด้วย dbt

ระยะเวลา 2 ชั่วโมง | PostgreSQL + dbt + Metabase + Docker

วัตถุประสงค์

- อธิบาย grain ของ fact table และเปรียบเทียบข้อมูลระดับ transaction, รายวัน และรายเดือนได้
- จำแนก measures เป็น Additive, Semi-additive และ Non-additive ได้อย่างเหมาะสม
- ใช้ dbt seed และ dbt models เตรียมข้อมูลหลายเดือนและร้านค้าในทุกจังหวัดได้
- สร้าง aggregate table ที่มี grain เป็นหนึ่งแถวต่อเดือนต่อ region ด้วย ref() ได้
- ใช้ dbt test ตรวจสอบ grain, row count, referential integrity และการกระทบยอดกับ fact table ได้
- สร้าง Roll-up, Drill-down และ CUBE models แล้วนำไปวิเคราะห์บน Metabase ได้

เครื่องมือและชุดข้อมูล

เครื่องมือ	หน้าที่ใน Lab
PostgreSQL	จัดเก็บ seed, models, aggregate table และ reporting views
dbt-postgres	seed, transform, test, document และแสดง lineage
Metabase	สร้างกราฟและ Dashboard สำหรับ Roll-up / Drill-down
Docker Compose	เปิด services postgres, dbt และ metabase
VS Code / Text editor	สร้างไฟล์ SQL และ YAML ในโครงการ dbt

- coffee_sales.csv - 3,000 รายการขาย ช่วงวันที่ 2024-07-01 ถึง 2024-07-31 (ใช้ชุดเดียวกับ Lab 4-5)
- province_region_mapping_v2.csv - 77 จังหวัด แบ่งเป็น 6 regions

ส่วนที่ 1 วิเคราะห์ Granularity และ Additivity

1.1 ระบุ grain ก่อนเขียน SQL

Grain คือคำอธิบายว่า 1 แถวของตารางแทนเหตุการณ์หรือสรุปข้อมูลในระดับใด ต้องกำหนด grain ก่อนเลือก dimension keys และ measures

Model / Query	Grain: หนึ่งแถวแทน...	คีย์ที่กำหนด grain
fct_sales	หนึ่งรายการสินค้าในใบเสร็จ หลังขยายเป็นเดือนเป้าหมาย	sale_key
rpt_sales_store_day	หนึ่งวันต่อหนึ่งร้าน	sale_date + store_key
agg_sales_region_month	หนึ่งเดือนต่อหนึ่ง region	month_start + region_key
rpt_sales_region_quarter	หนึ่งไตรมาสต่อหนึ่ง region	quarter_start + region_key

หลักตรวจ grain: หาก GROUP BY ใช้ month_start และ region_key
ผลลัพธ์ต้องมีได้ไม่เกินหนึ่งแถวต่อคู่ค่านี้ dbt test จะตรวจเงื่อนไขดังกล่าวอีกครั้ง

ส่วนที่ 2 เตรียม Lab Environment

2.1 เปิด services และสร้างฐานข้อมูล

```
CREATE DATABASE lab8;
```

2.2 สร้างโครงสร้างโครงการ dbt

```
dbt_root/
├── profiles.yml
└── dbt/
    ├── lab8/
    │   ├── dbt_project.yml
    │   ├── seeds/
    │   │   ├── coffee_sales.csv
    │   │   └── province_region_mapping_v2.csv
    │   ├── models/
    │   │   ├── staging/
    │   │   │   ├── stg_coffee_sales.sql
    │   │   │   └── stg_province_region_mapping.sql
    │   │   ├── intermediate/
    │   │   │   ├── int_invoice_store_map.sql
    │   │   │   └── int_sales_expanded.sql
    │   │   ├── marts/
    │   │   │   ├── dim_*.sql
    │   │   │   ├── fct_sales.sql
    │   │   │   ├── aggregates/
    │   │   │   │   └── agg_sales_region_month.sql
    │   │   │   └── reporting/
    │   │   │       ├── rpt_sales_store_day.sql
    │   │   │       ├── rpt_sales_region_quarter.sql
    │   │   │       ├── rpt_sales_province_quarter.sql
    │   │   │       └── rpt_sales_staff_year_cube.sql
    │   │   └── schema.yml
    │   └── tests/
    │       ├── assert_all_provinces_have_store.sql
    │       ├── assert_fact_expected_row_count.sql
    │       ├── assert_summary_grain.sql
    │       └── assert_summary_reconciles.sql
```

คัดลอก coffee_sales.csv และ province_region_mapping_v2.csv ไปไว้ใน dbt/lab8/seeds/ โดยคงชื่อไฟล์เดิม

2.3 กำหนด profile และ project configuration

dbt_root/profiles.yml

```
lab8:
  target: dev
  outputs:
    dev:
      type: postgres
      host: postgres
      port: 5432
      user: dw_user
      password: dw_pass
      dbname: lab8
      schema: dbt
      threads: 4
```

dbt/lab8/dbt_project.yml

```
name: lab8
version: '1.0.0'
config-version: 2

profile: lab8

model-paths: ["models"]
seed-paths: ["seeds"]
test-paths: ["tests"]

target-path: "target"
clean-targets:
  - "target"
  - "dbt_packages"

models:
  lab8:
    staging:
      +materialized: view
      +schema: staging
    intermediate:
      +materialized: view
      +schema: intermediate
    marts:
      +materialized: table
      +schema: marts
    aggregates:
      +materialized: table
      +schema: aggregates
    reporting:
      +materialized: view
      +schema: reporting

seeds:
  lab8:
    +schema: raw
```

```

coffee_sales:
+column_types:
  sale_id: integer
  sale_date: date
  birth_year: integer
  unit_price: numeric(12,2)
  quantity: integer
  revenue: numeric(14,2)
  points_redeemed: integer
province_region_mapping_v2:
+column_types:
  province_name: varchar(100)
  region_name: varchar(50)

```

ตรวจการเชื่อมต่อ

```

docker exec -it dw_dbt bash
cd lab8
dbt debug

```

ผลที่ได้: **Connection test: OK** และ **All checks passed**

ส่วนที่ 3 โหลด Seed และสร้าง Staging Models

3.1 โหลด CSV ด้วย dbt seed

ภายใน container dw_dbt

```

dbt seed --full-refresh
dbt show --select coffee_sales
dbt show --select province_region_mapping_v2

```

ผลที่คาดหวัง: coffee_sales มี 3,000 แถว และ province_region_mapping_v2 มี 77 แถว

3.2 ทำความสะอาดข้อมูลยอดขาย

models/staging/stg_coffee_sales.sql

```

select
  cast(sale_id as integer) as sale_id,
  trim(invoice_number) as invoice_number,
  cast(sale_date as date) as sale_date,
  trim(customer_code) as customer_code,
  trim(customer_name) as customer_name,
  trim(gender) as gender,
  cast(birth_year as integer) as birth_year,
  trim(product_code) as product_code,
  trim(product_name) as product_name,
  lower(trim(category)) as category,
  trim(size) as size,
  cast(unit_price as numeric(12,2)) as unit_price,
  cast(quantity as integer) as quantity,
  cast(revenue as numeric(14,2)) as revenue,
  trim(store_code) as store_code,
  trim(store_name) as store_name,
  trim(province) as province,
  trim(staff_code) as staff_code,
  trim(staff_name) as staff_name,
  trim(position) as position,
  nullif(trim(promo_code), '') as promo_code,
  nullif(trim(promo_desc), '') as promo_desc,

```

```
coalesce(cast(points_redeemed as integer), 0) as points_redeemed
from {{ ref('coffee_sales') }}
```

models/staging/stg_province_region_mapping.sql

```
select
  trim(province_name) as province_name,
  trim(region_name) as region_name
from {{ ref('province_region_mapping_v2') }}
```

รันเฉพาะ staging layer

```
dbt run --select path:models/staging
dbt show --select stg_coffee_sales --limit 5
```

ส่วนที่ 4 สร้าง Dimensions และ Fact Table ด้วย dbt

ส่วนนี้สร้าง Snowflake Schema จาก Lab 5 ใหม่ใน project แยก โดยใช้ deterministic surrogate keys จาก md5() เพื่อให้รันซ้ำแล้วได้คีย์เดิม

4.1 Dimensions พื้นฐานจาก coffee_sales

models/marts/dim_customer.sql

```
select distinct
  md5(customer_code) as customer_key,
  customer_code,
  customer_name,
  gender,
  birth_year
from {{ ref('stg_coffee_sales') }}
```

models/marts/dim_category.sql

```
select distinct
  md5(category) as category_key,
  category as category_name
from {{ ref('stg_coffee_sales') }}
```

models/marts/dim_product.sql

```
select distinct
  md5(s.product_code || '|' || s.size) as product_key,
  s.product_code,
  s.product_name,
  c.category_key,
  s.size,
  s.unit_price
from {{ ref('stg_coffee_sales') }} s
join {{ ref('dim_category') }} c
on s.category = c.category_name
```

models/marts/dim_position.sql

```
select distinct
  md5(position) as position_key,
  position as position_name
from {{ ref('stg_coffee_sales') }}
```

models/marts/dim_staff.sql

```
select distinct
  md5(s.staff_code) as staff_key,
  s.staff_code,
  s.staff_name,
  p.position_key
from {{ ref('stg_coffee_sales') }} s
join {{ ref('dim_position') }} p
on s.position = p.position_name
```

models/marts/dim_promotion.sql

```
select distinct
  md5(promo_code) as promo_key,
  promo_code,
  promo_desc
from {{ ref('stg_coffee_sales') }}
where promo_code is not null
```

4.2 Geography Dimensions จาก mapping v2

models/marts/dim_region.sql

```
select distinct
  md5(region_name) as region_key,
  region_name
from {{ ref('stg_province_region_mapping') }}
```

models/marts/dim_province.sql

```
select distinct
  md5(m.province_name) as province_key,
  m.province_name,
  r.region_key
from {{ ref('stg_province_region_mapping') }} m
join {{ ref('dim_region') }} r
  on m.region_name = r.region_name
```

4.3 สร้างร้านให้ครบทุกจังหวัดแบบ Declarative

โมเดล dim_store รักษาร้านเดิม 3 แห่ง และสร้างร้านจำลองอีก 1 แห่งเฉพาะจังหวัดที่ยังไม่มีร้าน โดยไม่ใช่

INSERT หรือ UPDATE

models/marts/dim_store.sql

```
with existing_stores as (
  select distinct
    s.store_code,
    s.store_name,
    p.province_key,
    p.province_name
  from {{ ref('stg_coffee_sales') }} s
  join {{ ref('dim_province') }} p
    on s.province = p.province_name
),

provinces_without_store as (
  select
    p.province_key,
    p.province_name,
    row_number() over (order by p.province_name) as generated_num
  from {{ ref('dim_province') }} p
  left join existing_stores e
    on p.province_key = e.province_key
  where e.province_key is null
),

all_stores as (
  select
    md5(store_code) as store_key,
    store_code,
    store_name,
    province_key,
    false as is_generated
  from existing_stores

  union all

  select
```

```

md5('GENERATED' || province_name) as store_key,
'ST' || lpad((generated_num + 3)::text, 3, '0') as store_code,
province_name || ' Center' as store_name,
province_key,
true as is_generated
from provinces_without_store
)

select * from all_stores

```

การแก้ไขจาก SQL เดิม: ไฟล์ข้อมูลจริงใช้รหัสร้าน ST01, ST02 และ ST03 ไม่ใช่ ST001–ST003
โมเดลนี้จึงตรวจร้านเดิมจากข้อมูลด้วย join และใช้ is_generated แทนการ hard-code รายชื่อรหัสร้าน

4.4 สร้าง Date Dimension ครอบคลุม 9 เดือน

models/marts/dim_date.sql

```

select
  to_char(d::date, 'YYYYMMDD')::integer as date_key,
  d::date as sale_date,
  extract(year from d)::integer as year,
  extract(quarter from d)::integer as quarter,
  extract(month from d)::integer as month,
  to_char(d, 'YYYY-MM') as year_month,
  extract(day from d)::integer as day_of_month
from generate_series(
  date '2024-07-01',
  date '2025-03-31',
  interval '1 day'
) as g(d)

```

4.5 กำหนดร้านเป้าหมายให้แต่ละ Invoice

Invoice เดียวกันต้องอยู่ร้านเดียวกันทุก line item จึงสร้าง mapping ที่ grain = หนึ่งแถวต่อ invoice_number
ก่อนขยายข้อมูลเป็นหลายเดือน

models/intermediate/int_invoice_store_map.sql

```

with generated_stores as (
  select
    store_code,
    row_number() over (order by store_code) as store_num,
    count(*) over () as store_count
  from {{ ref('dim_store') }}
  where is_generated
),

invoices as (
  select distinct invoice_number
  from {{ ref('stg_coffee_sales') }}
)

select
  i.invoice_number,
  g.store_code as target_store_code
from invoices i
join generated_stores g
on g.store_num = (
  (hashtext(i.invoice_number)::bigint & 2147483647)
  % g.store_count
) + 1

```

4.6 ขยายข้อมูลเดือน Jul 2024 ถึง Mar 2025

`models/intermediate/int_sales_expanded.sql`

```
with months as (
  select month_start::date
  from generate_series(
    date '2024-07-01',
    date '2025-03-01',
    interval '1 month'
  ) as g(month_start)
),

expanded as (
  select
    s.*,
    m.month_start,
    map.target_store_code,
    make_date(
      extract(year from m.month_start)::integer,
      extract(month from m.month_start)::integer,
      least(
        extract(day from s.sale_date)::integer,
        extract(day from (
          date_trunc('month', m.month_start)
          + interval '1 month - 1 day'
        ))::integer
      )
    ) as target_sale_date
  from {{ ref('stg_coffee_sales') }} s
  cross join months m
  join {{ ref('int_invoice_store_map') }} map
  on s.invoice_number = map.invoice_number
)

select
  md5(sale_id::text || '|' || month_start::text) as sale_key,
  sale_id as source_sale_id,
  case
    when month_start = date '2024-07-01' then invoice_number
    else invoice_number || '_' || to_char(month_start, 'YYYYMM')
  end as invoice_number,
  target_sale_date as sale_date,
  customer_code,
  product_code,
  size,
  target_store_code as store_code,
  staff_code,
  promo_code,
  quantity,
  revenue,
  points_redeemed
from expanded
```

วันที่ 29–31: เมื่อขยายไปยังเดือนที่มีจำนวนวันน้อยกว่า โมเดลใช้วันสุดท้ายของเดือนเป้าหมาย เช่น 31 มกราคมจะกลายเป็น 28 กุมภาพันธ์ 2025

4.7 สร้าง Fact Table

`models/marts/fct_sales.sql`

```
select
  x.sale_key,
  x.source_sale_id,
  x.invoice_number,
  d.date_key,
  c.customer_key,
  p.product_key,
```



```

st.store_key,
sf.staff_key,
pr.promo_key,
x.quantity,
x.revenue,
x.points_redeemed
from {{ ref('int_sales_expanded') }} x
join {{ ref('dim_date') }} d
  on x.sale_date = d.sale_date
join {{ ref('dim_customer') }} c
  on x.customer_code = c.customer_code
join {{ ref('dim_product') }} p
  on x.product_code = p.product_code
  and x.size = p.size
join {{ ref('dim_store') }} st
  on x.store_code = st.store_code
join {{ ref('dim_staff') }} sf
  on x.staff_code = sf.staff_code
left join {{ ref('dim_promotion') }} pr
  on x.promo_code = pr.promo_code

```

Grain ของ fct_sales: หนึ่งแถวต่อรายการขายเดิมต่อเดือนเป้าหมาย โดย sale_key ต้องไม่ซ้ำ และคาดหวัง $3,000 \times 9 = 27,000$ แถว

ส่วนที่ 5 สํารวจ Granularity และสร้าง Summary Table

5.1 Transaction-level

สังเกตว่า invoice_number หนึ่งใบอาจมีหลายแถว เพราะแต่ละแถวแทนสินค้าหนึ่งรายการในใบเสร็จ

5.2 Daily sales per store

[models/reporting/rpt_sales_store_day.sql](#)

```

select
  d.sale_date,
  s.store_key,
  s.store_code,
  s.store_name,
  sum(f.revenue) as daily_revenue,
  sum(f.quantity) as daily_quantity,
  count(distinct f.invoice_number) as invoice_count
from {{ ref('fct_sales') }} f
join {{ ref('dim_date') }} d
  on f.date_key = d.date_key
join {{ ref('dim_store') }} s
  on f.store_key = s.store_key
group by
  d.sale_date,
  s.store_key,
  s.store_code,
  s.store_name

```

5.3 Monthly sales per region: Aggregate Table

โมเดลนี้ materialized เป็น table ตาม dbt_project.yml เพื่อเก็บผลรวมที่ query บ่อย ลดการ join fact กับ geography dimensions ซ้ำทุกครั้ง

[models/aggregates/agg_sales_region_month.sql](#)

```

{{
  config(
    indexes=[
      {'columns': ['month_start', 'region_key'], 'unique': true}
    ]
  )
}}

```

```

    ]
  )
}}

select
  date_trunc('month', d.sale_date)::date as month_start,
  r.region_key,
  r.region_name,
  sum(f.revenue) as total_revenue,
  sum(f.quantity) as total_quantity,
  sum(f.points_redeemed) as total_points_redeemed,
  count(distinct f.invoice_number) as invoice_count,
  count(*) as line_item_count
from {{ ref('fct_sales') }} f
join {{ ref('dim_date') }} d
  on f.date_key = d.date_key
join {{ ref('dim_store') }} s
  on f.store_key = s.store_key
join {{ ref('dim_province') }} p
  on s.province_key = p.province_key
join {{ ref('dim_region') }} r
  on p.region_key = r.region_key
group by
  date_trunc('month', d.sale_date)::date,
  r.region_key,
  r.region_name

```

Grain ของ Aggregate: หนึ่งแถวต่อ month_start + region_key ส่วน region_name เป็น descriptive attribute ไม่ใช่ตัวกำหนด grain

5.4 Table หรือ View?

Materialization	ข้อดี	ข้อควรพิจารณา	ใช้ใน Lab
view	ข้อมูลใหม่ตาม upstream ทุกครั้งและไม่ซ้ำพื้นที่จัดเก็บ	query ซับซ้อนอาจช้าจากการคำนวณซ้ำ	staging, intermediate, reporting
table	อ่านเร็วและสร้าง index ได้	ต้อง dbt run/build เพื่อ refresh	dimensions, fact, aggregate
incremental	ประมวลผลเฉพาะข้อมูลใหม่	ต้องกำหนด unique_key และกลยุธรีรับข้อมูลย้อนหลัง	หัวข้อต่อยอดหลัง Lab

ส่วนที่ 6 กำหนด Tests และรัน Pipeline

6.1 Generic Tests และ Documentation

[models/schema.yml](#)

```

version: 2

seeds:
- name: coffee_sales
  description: ข้อมูลขายเดือนกรกฎาคม 2024 จำนวน 3,000 line items
  columns:
  - name: sale_id
    tests: [not_null, unique]

```

```

- name: province_region_mapping_v2
description: Mapping 77 จังหวัดไปยัง 6 regions
columns:
  - name: province_name
    tests: [not_null, unique]
  - name: region_name
    tests: [not_null]

models:
- name: stg_coffee_sales
description: Cleaned sales; grain คือหนึ่งแถวต่อ sale_id
columns:
  - name: sale_id
    tests: [not_null, unique]
  - name: invoice_number
    tests: [not_null]
  - name: province
    tests: [not_null]

- name: dim_region
columns:
  - name: region_key
    tests: [not_null, unique]
  - name: region_name
    tests: [not_null, unique]

- name: dim_province
columns:
  - name: province_key
    tests: [not_null, unique]
  - name: province_name
    tests: [not_null, unique]
  - name: region_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_region')
          field: region_key

- name: dim_store
columns:
  - name: store_key
    tests: [not_null, unique]
  - name: store_code
    tests: [not_null, unique]
  - name: province_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_province')
          field: province_key

- name: dim_date
columns:
  - name: date_key
    tests: [not_null, unique]
  - name: sale_date
    tests: [not_null, unique]

- name: fct_sales
description: Fact ที่ขยายข้อมูลเดิมเป็น 9 เดือน
columns:
  - name: sale_key
    tests: [not_null, unique]
  - name: date_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_date')
          field: date_key
  - name: store_key
    tests:

```

```

- not_null
- relationships:
  to: ref('dim_store')
  field: store_key
- name: revenue
  tests: [not_null]

- name: agg_sales_region_month
  description: Summary table; grain คือหนึ่งเดือนต่อหนึ่ง region
  columns:
    - name: month_start
      tests: [not_null]
    - name: region_key
      tests:
        - not_null
        - relationships:
          to: ref('dim_region')
          field: region_key
    - name: total_revenue
      tests: [not_null]
    - name: total_quantity
      tests: [not_null]

```

6.2 Singular Tests สำหรับกฎทางธุรกิจ

tests/assert_all_provinces_have_store.sql

```

select p.*
from {{ ref('dim_province') }} p
left join {{ ref('dim_store') }} s
  on p.province_key = s.province_key
where s.store_key is null

```

tests/assert_fact_expected_row_count.sql

```

select count(*) as actual_rows
from {{ ref('fct_sales') }}
having count(*) <> 27000

```

tests/assert_summary_grain.sql

```

select
  month_start,
  region_key,
  count(*) as duplicate_rows
from {{ ref('agg_sales_region_month') }}
group by month_start, region_key
having count(*) > 1

```

tests/assert_summary_reconciles.sql

```

with fact_total as (
  select
    sum(revenue) as revenue,
    sum(quantity) as quantity,
    sum(points_redeemed) as points
  from {{ ref('fct_sales') }}
),

summary_total as (
  select
    sum(total_revenue) as revenue,
    sum(total_quantity) as quantity,
    sum(total_points_redeemed) as points
  from {{ ref('agg_sales_region_month') }}
)

select
  f.revenue as fact_revenue,
  s.revenue as summary_revenue,
  f.quantity as fact_quantity,
  s.quantity as summary_quantity,

```

```
f.points as fact_points,
s.points as summary_points
from fact_total f
cross join summary_total s
where abs(f.revenue - s.revenue) > 0.01
or f.quantity <> s.quantity
or f.points <> s.points
```

Singular test ต้องคืนค่า 0 แถวจึงจะผ่าน หากมีผลลัพธ์ แถวนั้นคือหลักฐานของปัญหาที่ต้องตรวจสอบ

6.3 รัน Pipeline ด้วยคำสั่งเดียว

```
dbt build
```

dbt build จะ seed, สร้าง models และรัน tests ตาม dependency graph ที่เกิดจาก ref()

ตรวจจำนวนแถวสำคัญ

```
dbt show --inline "
select 'dim_region' as model, count(*) as row_count from {{ ref('dim_region') }}
union all select 'dim_province', count(*) from {{ ref('dim_province') }}
union all select 'dim_store', count(*) from {{ ref('dim_store') }}
union all select 'dim_date', count(*) from {{ ref('dim_date') }}
union all select 'fct_sales', count(*) from {{ ref('fct_sales') }}
union all select 'agg_sales_region_month', count(*)
from {{ ref('agg_sales_region_month') }}
" --limit 6
```

Model	จำนวนที่คาดหวัง	เหตุผล
dim_region	6	6 regions ใน mapping v2
dim_province	77	หนึ่งแถวต่อจังหวัด
dim_store	77	หนึ่งร้านต่อจังหวัดหลังรวมร้านเดิมและร้านจำลอง
dim_date	274	2024-07-01 ถึง 2025-03-31
fct_sales	27,000	3,000 แถว × 9 เดือน
agg_sales_region_month	54	9 เดือน × 6 regions เมื่อทุก region มียอดขาย

ส่วนที่ 7 สร้าง OLAP Models

7.1 Roll-up: เดือน → ไตรมาส

Roll-up ใช้ aggregate table รายเดือนเป็นต้นทาง จึงไม่ต้องอ่าน fact table และ join dimensions ใหม่

models/reporting/rpt_sales_region_quarter.sql

```
select
  date_trunc('quarter', month_start)::date as quarter_start,
  region_key,
  region_name,
  sum(total_revenue) as quarter_revenue,
  sum(total_quantity) as quarter_quantity,
  sum(total_points_redeemed) as quarter_points_redeemed,
  sum(invoice_count) as quarter_invoice_count
from {{ ref('agg_sales_region_month') }}
group by
  date_trunc('quarter', month_start)::date,
  region_key,
  region_name
```

ข้อสังเกต: revenue, quantity และ points_redeemed เป็น additive จึง SUM ต่อได้
ส่วนค่าเฉลี่ยหรือเปอร์เซ็นต์ต้องกลับไปคำนวณจาก numerator และ denominator ที่เก็บไว้

7.2 Drill-down: Region → Province

Aggregate รายเดือนต่อ region ไม่มี province_key จึง drill-down จากตารางนั้นโดยตรงไม่ได้ ต้องอ่าน fact table แล้ว group ที่ grain ละเอียดกว่า

models/reporting/rpt_sales_province_quarter.sql

```
select
  date_trunc('quarter', d.sale_date)::date as quarter_start,
  r.region_key,
  r.region_name,
  p.province_key,
  p.province_name,
  sum(f.revenue) as province_revenue,
  sum(f.quantity) as province_quantity,
  count(distinct f.invoice_number) as invoice_count
from {{ ref('fct_sales') }} f
join {{ ref('dim_date') }} d
  on f.date_key = d.date_key
join {{ ref('dim_store') }} s
  on f.store_key = s.store_key
join {{ ref('dim_province') }} p
  on s.province_key = p.province_key
join {{ ref('dim_region') }} r
  on p.region_key = r.region_key
group by
  date_trunc('quarter', d.sale_date)::date,
  r.region_key,
  r.region_name,
  p.province_key,
  p.province_name
```

7.3 CUBE: Year × Staff

models/reporting/rpt_sales_staff_year_cube.sql

```
select
  d.year,
  f.staff_key,
  grouping(d.year) as is_all_years,
  grouping(f.staff_key) as is_all_staff,
  sum(f.revenue) as total_revenue,
  sum(f.quantity) as total_quantity
from {{ ref('fct_sales') }} f
join {{ ref('dim_date') }} d
  on f.date_key = d.date_key
group by cube(d.year, f.staff_key)
```

is_all_years	is_all_staff	ความหมายของแถว
0	0	ยอดของ staff แต่ละคนในแต่ละปี
0	1	ยอดรวม staff ทุกคนในปีนั้น
1	0	ยอดรวมทุกปีของ staff คนนั้น
1	1	Grand total ทุกปีและทุก staff

สร้างและตรวจ OLAP models

```
dbt run --select path:models/reporting
dbt show --select rpt_sales_region_quarter --limit 30
dbt show --select rpt_sales_province_quarter --limit 30
dbt show --select rpt_sales_staff_year_cube --limit 30
```

ส่วนที่ 8 สร้าง dbt Documentation และตรวจ Lineage

```
dbt docs generate
dbt docs serve --host 0.0.0.0 --port 8080
```

เปิด <http://localhost:28088> แล้วค้นหา agg_sales_region_month จากนั้นเปิด Lineage Graph

- ตรวจ upstream: seeds → staging → dimensions/intermediate → fct_sales
- ตรวจ downstream: agg_sales_region_month → rpt_sales_region_quarter
- เปิดแท็บ Tests เพื่อดู generic tests และ singular tests ที่เชื่อมกับโมเดล

เหตุผลที่ใช้ dbt docs: คำอธิบาย grain, คอลัมน์, tests, compiled SQL และ dependency จาก ref()
 ถูกรวมไว้ในจุดเดียว ทำให้ตรวจเส้นทางของ summary table กลับไปยังข้อมูลต้นทางได้

ส่วนที่ 9 สร้าง Dashboard บน Metabase

9.1 เชื่อมต่อ PostgreSQL

ค่า	กำหนดเป็น
Database type	PostgreSQL
Host	postgres
Port	5432
Database name	lab8
Username / Password	dw_user / dw_pass

เปิด <http://localhost:23000> แล้วเพิ่มฐานข้อมูล จากนั้นสั่ง Sync database schema หากยังไม่เห็น dbt_aggregates และ dbt_reporting

9.2 สร้างคำถามสำหรับ Roll-up

1. เลือก dbt_reporting.rpt_sales_region_quarter
2. แสดง Line chart: X-axis = quarter_start, Y-axis = quarter_revenue และ Breakout = region_name
3. บันทึกชื่อ Revenue by Quarter and Region

9.3 สร้างคำถามสำหรับ Drill-down

1. เลือก dbt_reporting.rpt_sales_province_quarter
2. เพิ่ม filters quarter_start และ region_name
3. แสดง Bar chart: X-axis = province_name และ Y-axis = province_revenue
4. บันทึกชื่อ Revenue by Province

9.4 ประกอบ Dashboard

1. สร้าง Dashboard ชื่อ Sales Roll-up and Drill-down
2. เพิ่มการ์ด Revenue by Quarter and Region และ Revenue by Province
3. เพิ่ม Dashboard filters: Quarter และ Region แล้ว map เข้ากับทั้งสองการ์ด
4. เลือก region หนึ่งค่าเพื่อตรวจว่า Province chart แสดงรายละเอียดเฉพาะ region ที่เลือก
5. เพิ่มตารางจาก rpt_sales_staff_year_cube และใช้ is_all_years / is_all_staff แยกระดับ subtotal

สิ่งที่ต้องส่ง

รายการ	รูปแบบ
Dashboard	Screenshot ส่งใน Google Form